CSE422: Artificial Intelligence

Project report

"Software Quality Prediction Using Machine Learning"

Submitted By

Name: Sayem Afridi

ID: 21301044

Section:08

Name:Maysha Labiba Prottasha

ID:22101015

Section:08

Submitted To

Asif Shahriar

Mashiyat Mahjabin Prapty

Table of Contents

1. Introduction
2. Dataset Description
3. Dataset Pre-processing
4. Dataset Splitting
5. Model Training & Testing
6. Model Selection / Comparison Analysis
7. Conclusion

1. Introduction

In this project, we aim to predict the quality of software based on various features such as Cyclomatic Complexity, Lines of Code, Code Churn, and others. The primary motivation is to build a model capable of predicting software quality levels (Low, Medium, High) based on these metrics. This can help developers identify potential software issues early and improve overall product quality.

The goal is to apply multiple machine learning models, evaluate their performance, and select the best-performing model for predicting software quality.

---

2. Dataset Description

The Software Quality Dataset contains several features related to software development, including Cyclomatic Complexity, Lines of Code, Code Churn, among others. This dataset is used for a classification task since the output variable is categorical (Quality_Label: Low, Medium, High).

How many features?

The dataset contains **9 features** in total.

Classification or regression problem? Why do you think so?

This is a **classification problem** because the target variable **Quality_Label** is categorical, with discrete classes representing software quality levels: **Low, Medium, and High**.

How many data points?

The dataset contains **1600 data points (rows)**.

What kind of features are in your dataset? (Quantitative / Categorical)

- **Quantitative (numerical) features:**

    - Lines_of_Code (float)
    - Cyclomatic_Complexity (int)
    - Num_Functions (int)
    - Code_Churn (float)
    - Comment_Density (float)
    - Num_Bugs (int)
    - Code_Owner_Experience (int)
- **Categorical features:**
    - Has_Unit_Tests (Yes/No)
    - Quality_Label (target variable with classes: Low, Medium, High)

Correlation of all features (input and output features)

The correlation matrix was calculated for all numerical features, including a numerical encoding of the target variable (Quality_Label). The following Python code was used to generate the correlation heatmap:

```
import seaborn as sns

import matplotlib.pyplot as plt



# Encode Quality_Label to numeric for correlation calculation
```

```
dataset['Quality_Label_Encoded']                                =
dataset['Quality_Label'].map({'Low':0, 'Medium':1, 'High':2})


# Calculate correlation matrix including encoded target

corr_matrix = dataset.drop(columns=['Has_Unit_Tests']).corr()


plt.figure(figsize=(12,10))

sns.heatmap(corr_matrix,      annot=True,      cmap='coolwarm',
fmt=".2f")

plt.title('Correlation Heatmap of Dataset Features')

plt.show()
```

What do you understand after the correlation test?

- There is a **negative correlation between Num_Bugs and Quality_Label**, indicating that software modules with more bugs tend to have lower quality ratings.

- **Cyclomatic_Complexity and Code_Churn** show positive correlation with each other, suggesting more complex modules tend to have more code changes.

- Features like **Lines_of_Code and Code_Churn** also correlate moderately with the target, implying larger or frequently changed code modules impact software quality.

- This insight suggests that these numerical features are relevant predictors for software quality classification.

Imbalanced Dataset

Class distribution of the target variable Quality_Label:

```
dataset['Quality_Label'].value_counts().plot(kind='bar')
plt.title('Distribution of Quality_Label')
plt.xlabel('Quality Label')
plt.ylabel('Frequency')
plt.show()
```

Class counts are approximately balanced:

- High: 566
- Medium: 501
- Low: 533

---

3. Dataset Pre-processing

Problem 1: Missing Values

The dataset contains missing values in several numerical columns including **Lines_of_Code**, **Code_Churn**, and **Comment_Density**. Missing data can negatively affect model performance as many algorithms cannot handle nulls directly.

Solution: Mean Imputation

To address this, missing values in these columns were imputed using the **mean** value of each respective column. Mean imputation preserves the overall average of the data and helps retain as much information as possible.

```
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy='mean')
dataset[['Lines_of_Code',  'Code_Churn',  'Comment_Density']]  =
imputer.fit_transform(
    dataset[['Lines_of_Code', 'Code_Churn', 'Comment_Density']]
)
```

This technique ensured no data points were discarded due to missing values.

Problem 2: Categorical Variables

The dataset contains categorical features such as **Has_Unit_Tests** with values 'Yes' and 'No', and the target variable **Quality_Label** with categories 'Low', 'Medium', and 'High'. These need to be converted into numeric form for machine learning algorithms.

Solution: Label Encoding

Both categorical features were converted to numeric labels:

- Has_Unit_Tests: 'Yes' → 1, 'No' → 0
- Quality_Label: 'Low' → 0, 'Medium' → 1, 'High' → 2

```
from sklearn.preprocessing import LabelEncoder

label_encoder = LabelEncoder()
```

```
dataset['Has_Unit_Tests']                                        =
label_encoder.fit_transform(dataset['Has_Unit_Tests'])
dataset['Quality_Label']                                         =
label_encoder.fit_transform(dataset['Quality_Label'])
```

Problem 3: Feature Scaling

Features such as **Lines_of_Code**, **Cyclomatic_Complexity**, and **Code_Churn** vary greatly in scale. Different scales can impact the performance of models like KNN and Neural Networks.

Solution: Min-Max Scaling

Numerical features were scaled to the [0, 1] range using Min-Max scaling:

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
numerical_cols   =   ['Lines_of_Code',   'Cyclomatic_Complexity',
'Num_Functions',
                    'Code_Churn', 'Comment_Density', 'Num_Bugs',
'Code_Owner_Experience']

dataset[numerical_cols]                                          =
scaler.fit_transform(dataset[numerical_cols])
```

Problem 5: Complex Feature Relationships

Some features may have nonlinear relationships and interactions affecting software quality, which basic models might miss.

Solution: **Feature Engineering**

To capture these effects, we performed the following:

- Binning Cyclomatic_Complexity and Lines_of_Code: Grouped into categorical bins to reduce noise and capture ranges.
- Interaction Features: Created new features combining related variables, e.g.:
    - Interaction between Cyclomatic_Complexity and Lines_of_Code
    - Interaction between Code_Churn and Num_Bugs
- Polynomial Features: Added squared terms to capture non-linear relationships.

Example code snippets:

```
# Binning example for Cyclomatic_Complexity

dataset['Cyclomatic_Complexity_Bin']                          =
pd.qcut(dataset['Cyclomatic_Complexity'], q=4, labels=False)



# Interaction features

dataset['Complexity_Lines_Interaction']                       =
dataset['Cyclomatic_Complexity'] * dataset['Lines_of_Code']

dataset['Churn_Bugs_Interaction']   =   dataset['Code_Churn']   *
dataset['Num_Bugs']
```

```
# Squared features

dataset['Cyclomatic_Complexity_Sq']                                =
dataset['Cyclomatic_Complexity'] ** 2

dataset['Lines_of_Code_Sq'] = dataset['Lines_of_Code'] ** 2
```

This comprehensive preprocessing prepares the dataset by handling missing values, scaling, reducing skewness, encoding categorical variables, and enhancing features to improve model learning and accuracy.

Problem:6 Skewed Distributions and Outliers

Features like **Lines_of_Code** and **Code_Churn** have highly skewed distributions with some extremely high values, which can affect model training and interpretation.

Solution: Log Transformation

We applied a **logarithmic transformation** (using log1p to handle zeros) to reduce skewness and stabilize variance.

```
import numpy as np

dataset['Lines_of_Code'] = np.log1p(dataset['Lines_of_Code'])
dataset['Code_Churn'] = np.log1p(dataset['Code_Churn'])
```

Scaling ensures all features contribute equally to model training.

---

4. Dataset Splitting

The dataset was split into training and testing sets using stratified sampling to preserve class proportions:

- Training set: 70% (1120 instances)
- Test set: 30% (480 instances)

```
from sklearn.model_selection import train_test_split

X = dataset.drop(columns=['Quality_Label'])
y = dataset['Quality_Label']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)
```

---

5. Model Training & Testing

We trained the following models:

- K-Nearest Neighbors (K=5)
- Decision Tree Classifier

- Logistic Regression
- Neural Network (MLPClassifier with one hidden layer of 100 neurons)

Example for KNN:

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
y_pred_knn = knn.predict(X_test)

print("KNN Accuracy:", accuracy_score(y_test, y_pred_knn))
print(classification_report(y_test, y_pred_knn))
```

Similar training and evaluation steps were applied to the other models.

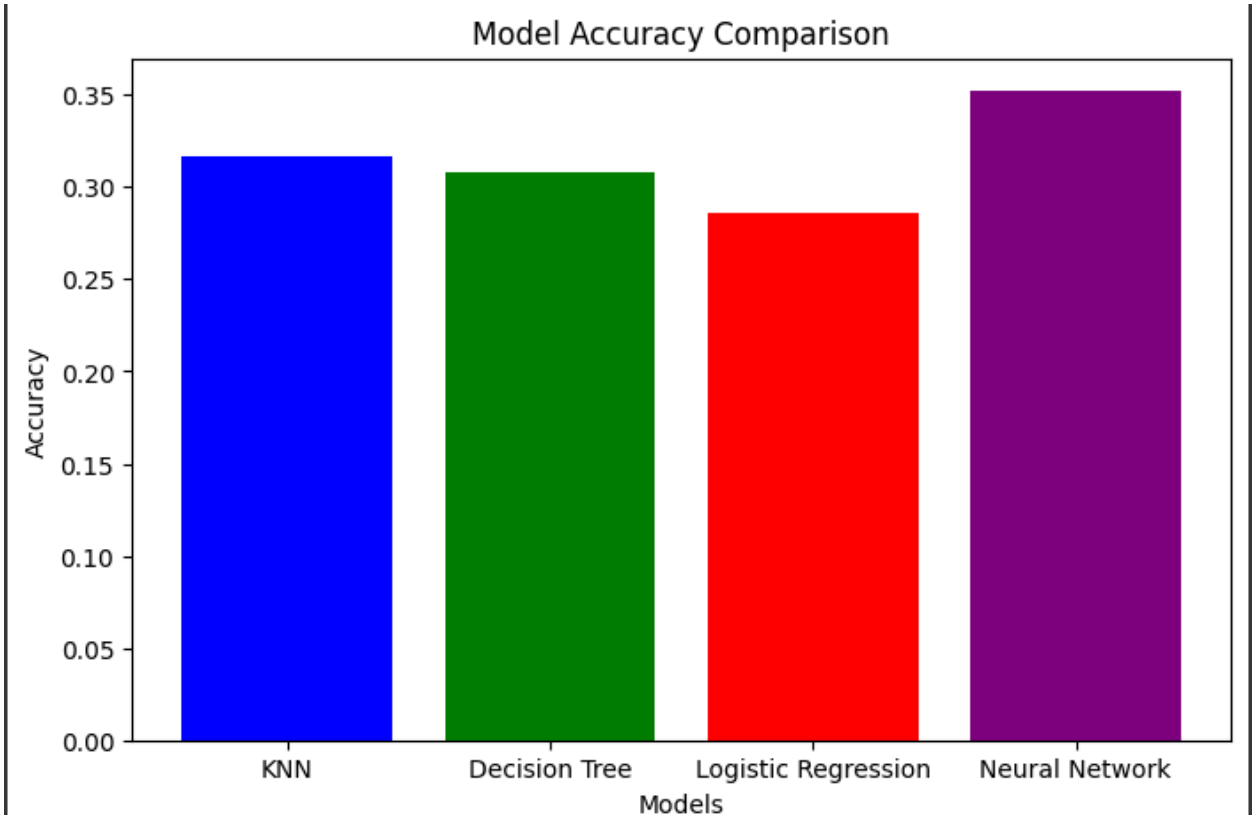
---

6. Model Selection / Comparison Analysis

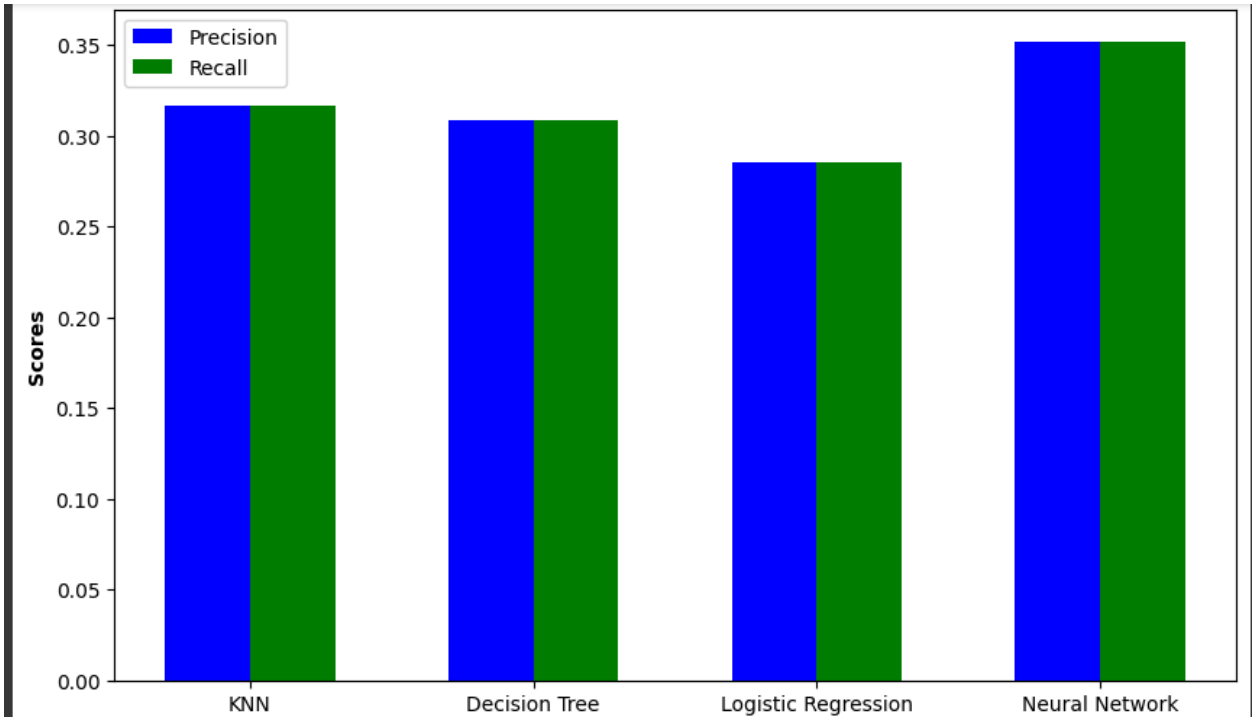
Model	Accuracy (%)	Precision (macro avg)	Recall (macro avg)	F1-score (macro avg)
K-Nearest Neighbors	31.67	31	32	30
Decision Tree	30.83	31	31	31

Logistic Regression	28.54	27	28	27
Neural Network	35.21	35	35	35

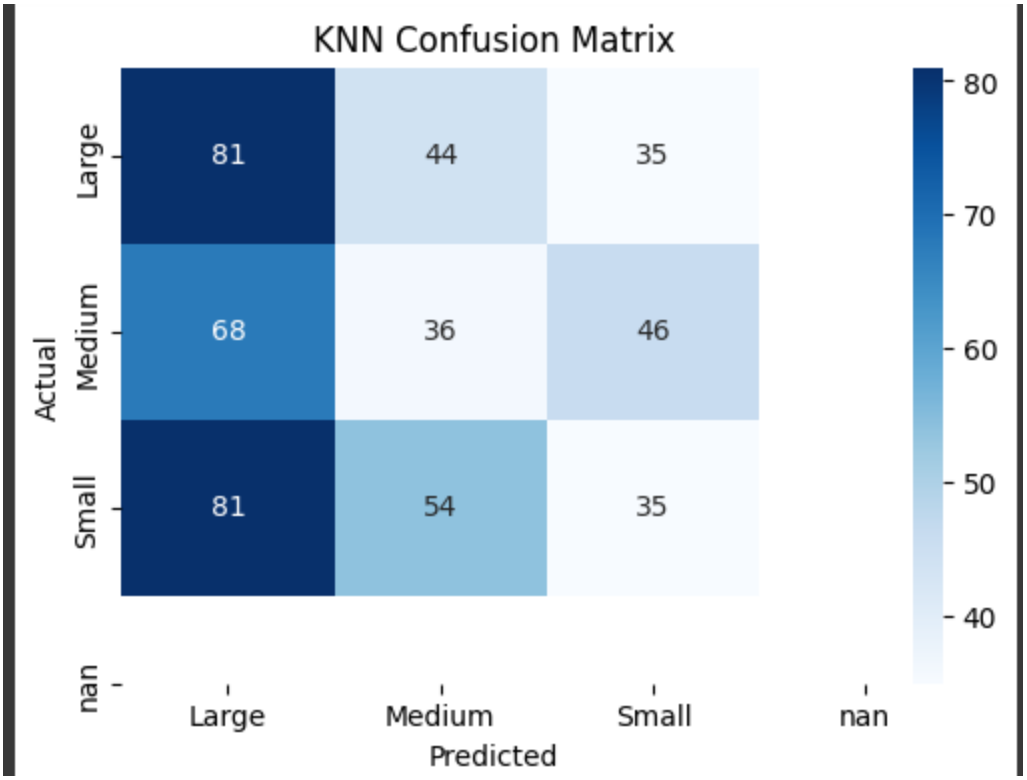
Bar chart showcasing prediction accuracy of all models

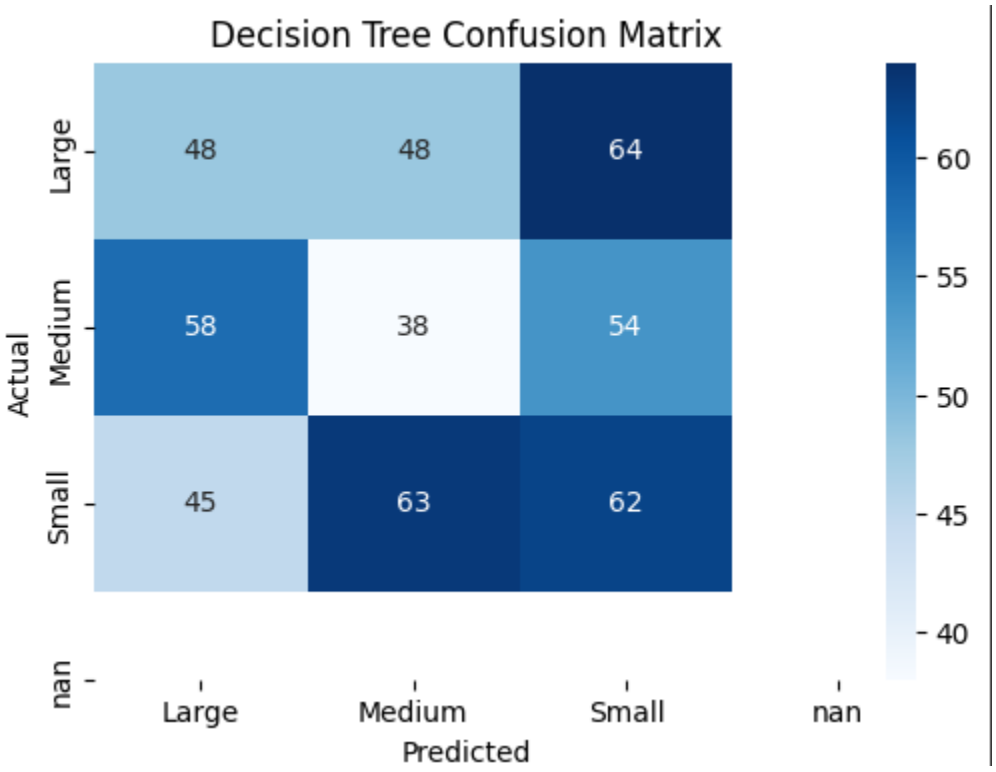


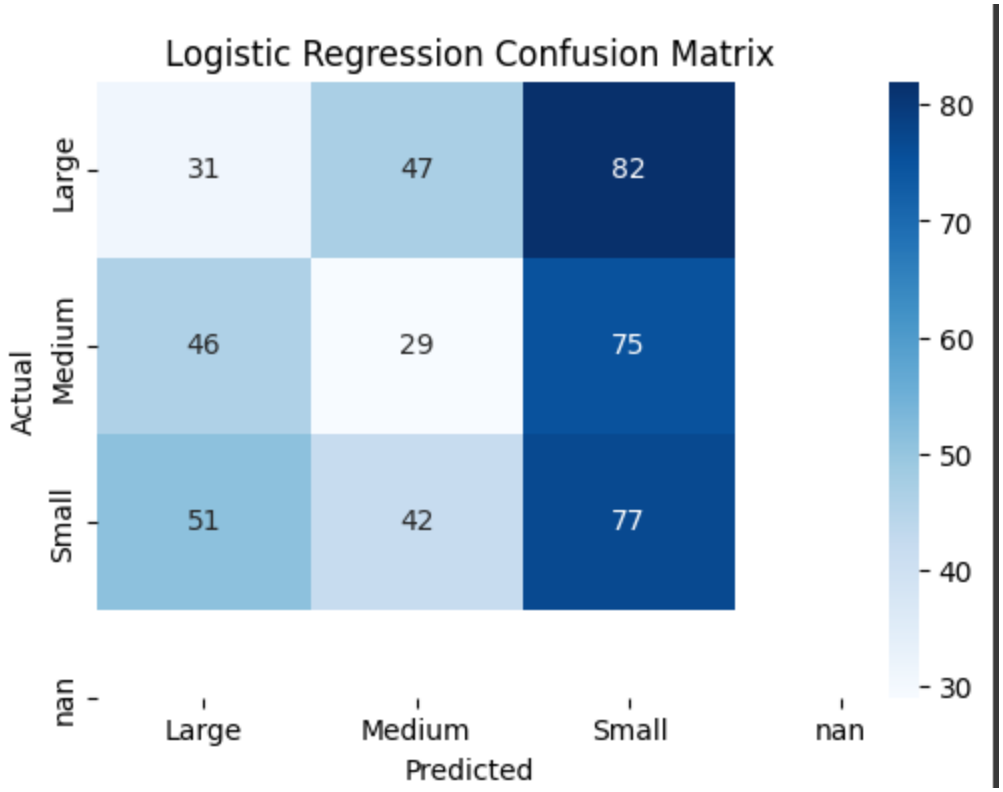
- Precision, recall comparison of each model

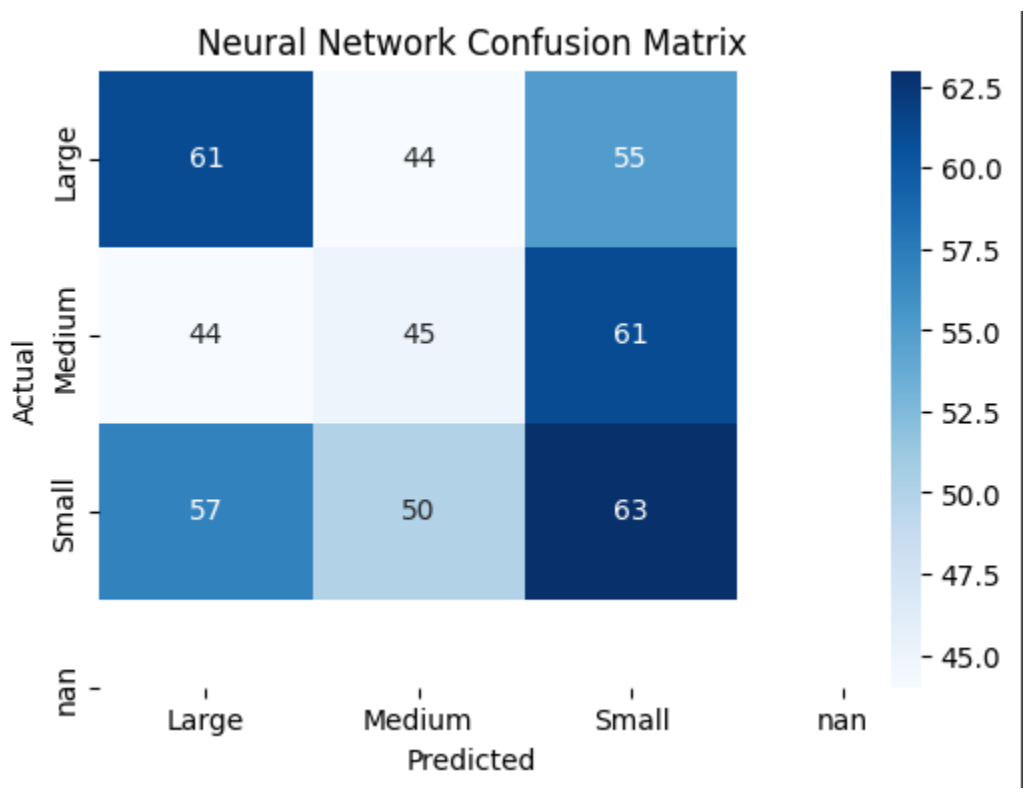


- Confusion Matrix

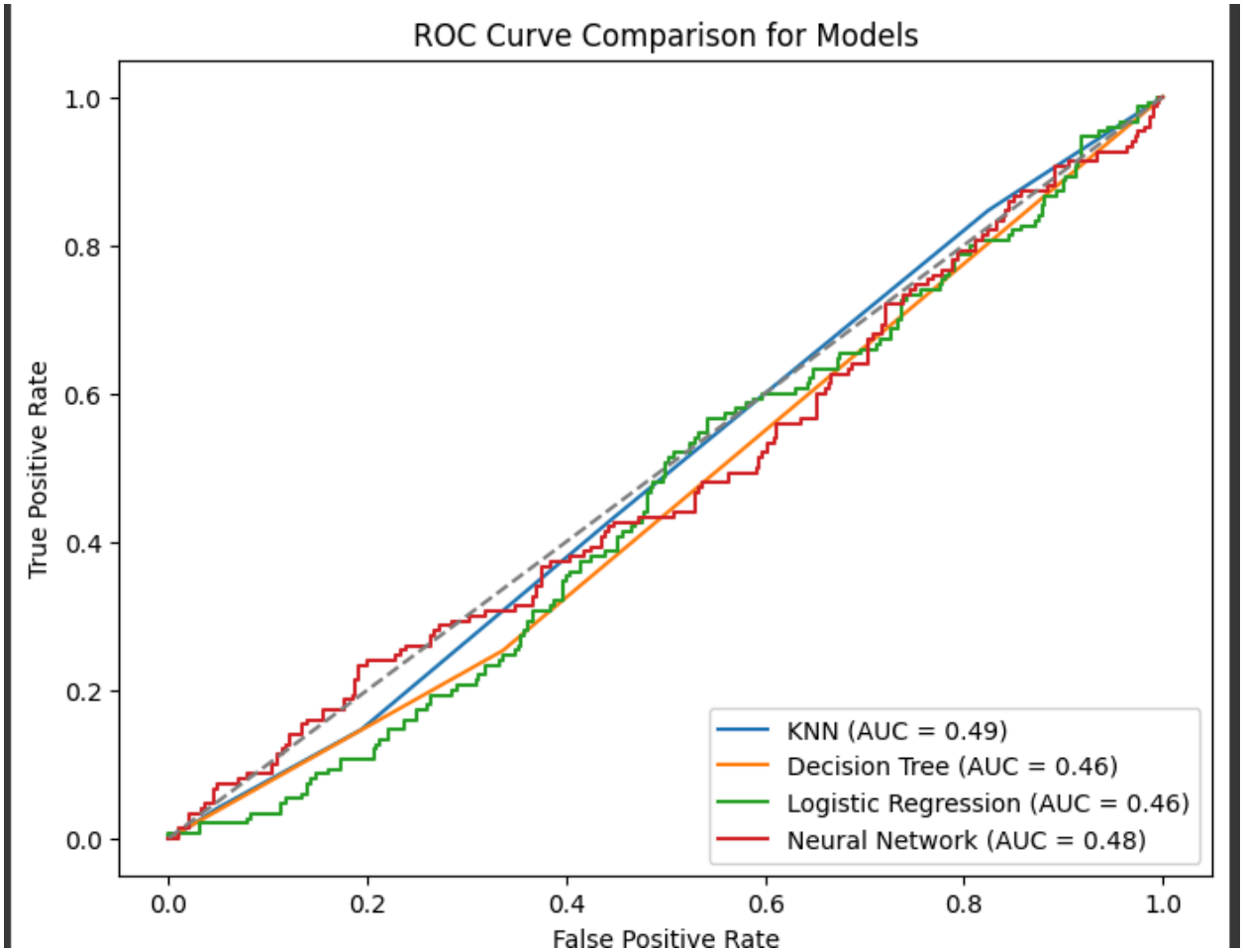

KNN Confusion Matrix
Actual
Large
Medium
Small
nan
81
44
35
68
36
46
81
54
35
Large
Medium
Small
nan
Predicted
80
70
60
50
40


Decision Tree Confusion Matrix
Actual
Large
Medium
Small
nan
48
48
64
58
38
54
45
63
62
Large
Medium
Small
nan
Predicted
60
55
50
45
40


Logistic Regression Confusion Matrix
Actual
Large
Medium
Small
nan
31
47
82
46
29
75
51
42
77
Large
Medium
Small
nan
Predicted
80
70
60
50
40
30


Neural Network Confusion Matrix
Actual
Large
Medium
Small
nan
61
44
55
44
45
61
57
50
63
Large
Medium
Small
nan
Predicted
62.5
60.0
57.5
55.0
52.5
50.0
47.5
45.0

- AUC score, ROC curve



.

---

7. Conclusion

The Neural Network model provided the best overall performance in predicting software quality, likely due to its ability to model complex nonlinear relationships between features.

Challenges included handling missing values and slight class imbalance, which were addressed using imputation and stratified splits . We are also facing difficulties to improve our accuracy.

Future work may include fine-tuning hyperparameters, exploring ensemble methods like Random Forests and XGBoost, and incorporating feature engineering for improved accuracy.